CardaLink — Team Workflow & Git Guide

Standard Operating Procedure for 5-Developer Modular Django Collaboration

1. Team Assignments & App Ownership

Each developer owns a single decoupled Django app. To avoid breaking dependencies, work must follow the clear data flow sequence outlined below.

Developer	App	Core Responsibility
Aby	`users`	Identity, Roles (Seller, Buyer, Auctioneer, Admin), Custom User Model
Sreedhar	`estates`	Estate registration, Harvest Batches, Quality certificates, Grading
Gouri	`auctions`	Auction scheduling, Lot index mapping, Real-time Bid engine
Vivek	`invoicing`	Invoice generation for won lots, Platform commission fees, Payment status
Riya	`assistant`	AI Cardamom cultivation advice, Chat query logging & FAQ knowledgebase

2. Day-by-Day Step-by-Step Workflow

■ Day 1: Aby builds users (Foundation)

Aby sets up the central User model. All other developers depend on this authentication layer.

```
git checkout main
git pull origin main
git checkout -b feature/users-auth

# Aby edits users/models.py, then runs:
python manage.py makemigrations users
python manage.py migrate

git add carda_link/users/
git commit -m "feat(users): Implement CustomUser model with roles"
git push origin feature/users-auth
```

■ Day 2: Sreedhar (estates) & Riya (assistant) Work in Parallel

Sreedhar links Estate to settings.AUTH_USER_MODEL. Riya builds the AI assistant logs simultaneously.

```
# Sreedhar pulls main to get Aby's User model:
git checkout main && git pull origin main
python manage.py migrate
git checkout -b feature/estate-management

python manage.py makemigrations estates
python manage.py migrate
git push origin feature/estate-management
```

■ Day 3: Gouri builds auctions (Dependent on estates)

Gouri links auction lots to harvest batches created by Sreedhar.

```
git checkout main && git pull origin main
python manage.py migrate # Applies Aby's users + Sreedhar's estates tables to Gouri's DB!
git checkout -b feature/auctions-bidding

# Gouri connects Lot to HarvestBatch (estates.HarvestBatch)
python manage.py makemigrations auctions
python manage.py migrate
git push origin feature/auctions-bidding
```

■ Day 4: Vivek builds invoicing (Dependent on auctions)

Vivek generates invoices automatically once Gouri's auction lots are marked as sold.

```
git checkout main && git pull origin main
python manage.py migrate # Vivek now has all tables from Aby, Sreedhar, Riya, and Gouri!
git checkout -b feature/invoicing-module

python manage.py makemigrations invoicing
python manage.py migrate
git push origin feature/invoicing-module
```

3. Handling Migration Conflicts

If two developers create migrations at the same time, Django will display: *Conflicting migrations detected*.

```
# Step 1: Run Django auto-merge command
python manage.py makemigrations --merge

# Step 2: Apply the newly created merge migration
python manage.py migrate

# Step 3: Commit and push the merge file to Git
git add carda_link//migrations/
git commit -m "fix(migrations): Resolve migration graph conflict"
git push origin
```

4. Golden Rules for Team Collaboration

- **Rule 1 (Daily Sync):** Always run git pull origin main and python manage.py migrate before starting work.
- **Rule 2 (Migration Integrity):** Never edit or delete a migration file that has already been merged into main.
- **Rule 3 (App Scoping):** Modify code ONLY inside your assigned app folder (e.g. carda_link/auctions/).
- **Rule 4 (No DB Commits):** Never commit local SQLite files, secrets, or .env files to Git.
- **Rule 5 (String FK References):** Use string foreign keys like 'estates.HarvestBatch' to prevent circular imports.